

本期课程 · 我们要做什么 · 能做到什么 · 为什么要做



做什么 · What

知识日报 Agent

cli-with-muyu group · NanoClaw

HN + RSS

Claude

WeChat

- 每天 09:00 北京时间自动跑
- HN Algolia search_by_date 端点
- 3 个 RSS 源抓取技术新闻
- Claude API 摘要 5 条结构化要闻
- add-wechat skill 真推手机微信



今天 90 min
把「我想要日报 Agent」
用 9 步协同精确落地

stage1-daily-news

spec · plan · code · review · ship



能做到 · Effect



开发真实工作系统



为什么 · Why

场景动机

NanoClaw 的产品形态不是一问一答，而是自动工作系统

元方法动机

Spec-kit + Superpowers 9步协同



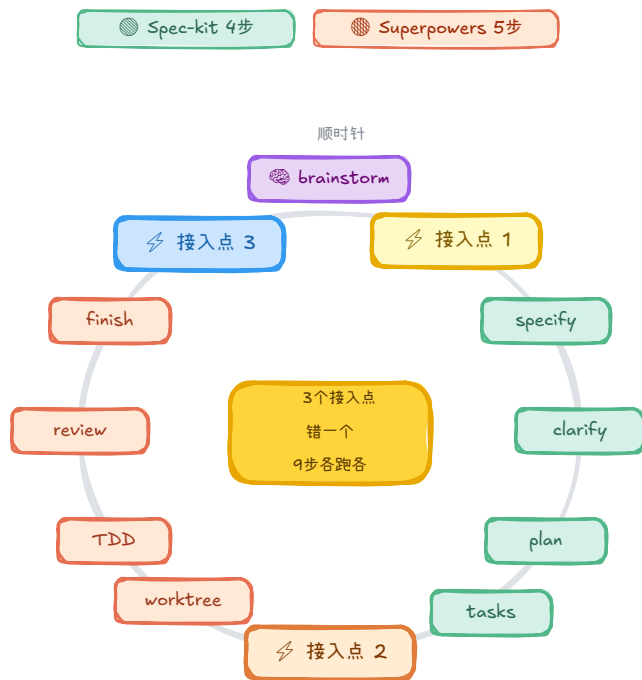
装了 NanoClaw 的
90% 人还在当聊天机器人用
——Agent 真实潜力被严重低估



从「模糊想法」到「真实工作系统」
通过自定义开发 · 永久改变工作方式

✨ Spec-kit 教 AI 看规范 · Superpowers 教 AI 做事 · 这是两个层级 ✨

Spec-kit ↔ Superpowers · 3个接入点 · 协作流程心智地图



跨天跨周 → 9步协同

⚡ 接入点 1 · brainstorming → Spec-kit specify

✅ 合格信号：MVP边界 + 风险 + 关键决策 三件齐全

❌ 反模式：还在发散就切 → clarify 追问不停

❌ 反模式：已写代码细节才切 → spec重组时丢信息

🔑 钥匙：“做X·MVP=ABC·不做DE·风险F·决策G” → 直接喂 specify

⚡ 接入点 2 · tasks.md → Superpowers 执行

✅ 合格信号：12-18条 2-5min任务 · 8并行组 · 入参出参清晰

❌ 反模式：直说“按spec干” → Superpowers re-planning一遍
tasks.md白拆·spec约束全丢

🔑 钥匙三词：Execute specs/<path>/tasks.md (防re-planning)

follow strict TDD per task one subagent/task

⚡ 接入点 3 · finish → Stage N+1 资产沉淀

✅ 合格信号：specs / 3件套 (spec·plan·review)

❌ 反模式：合branch后删specs/ → 后续Stage每次从零开始

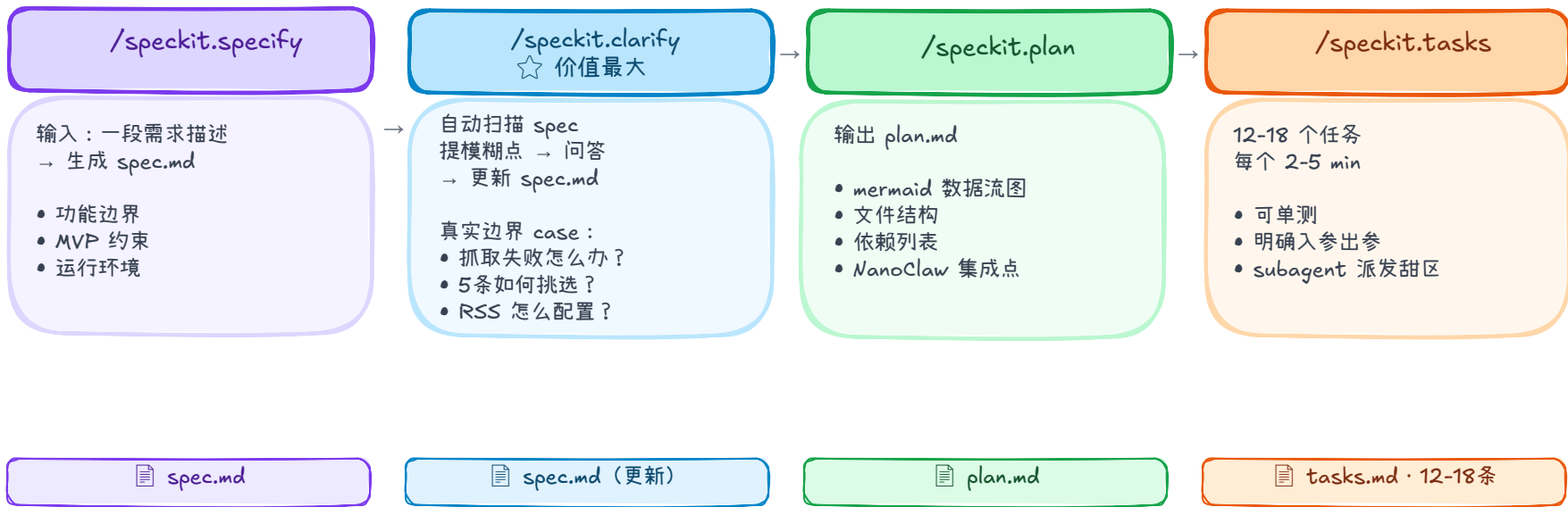
无工作记忆 → Compound Intelligence断链

🔑 钥匙：specs/ 不是文档是CI种子

© 两工具不在同一赛道：Spec-kit = 需求侧规范 (spec is source of truth) · Superpowers = 执行侧编排 (workflow is source of truth)

3个接入点找准 → 9步协同才能 1+1>2, 错一个 → 各跑各的效率倒挂

📄 需求侧 · Spec-kit 4 步 从「我想要」到 task 清单



💡 spec.md 不是文档 · 是给 LLM 看的规范契约——它精确，subagent 才能严格执行

⚡ 上面 4 步走完 · LLM 才知道你到底要什么

执行侧 · Superpowers 5 步 · 一句 handoff prompt → 12-18 subagent 自动派发

Step 6 🌿 worktree 隔离

git worktree add 切独立分支
物理隔离·不污染主分支
多 worktree 并发的基础

Step 7 ⚡ TDD subagent 派发

一句 handoff → 12-18 subagent
自动派发
每个严格 TDD:
先写 vitest → 🟡 RED
→ 实现 → 🟢 GREEN
→ refactor

Step 8 🔍 code review

"floor-raiser not ceiling-raiser"
命中率 30-40% (真实数据)
blocker 必修 major 建议
≠ 替代人工 review

Step 9 🏁 finish branch

commit + merge to main
删 worktree + 删 branch
打 v2.1.0-daily-news tag
一气呵成

"Execute tasks.md using the Superpowers workflow..." — 一句话桥接 spec-kit → Superpowers

🌿 worktree 物理隔离

- git worktree add -b feat/task
- 物理隔离·不影响主分支
- 多 worktree 同时运行
互不污染
- Stage 3: 并发 4-8 个

🔗 handoff prompt · 一句话桥接

必须明告「plan 已存在」
→ 防 Superpowers 双重 planning
必须列出「已 done 步骤」
→ 防重新发明轮子

桥接: tasks.md
→ Superpowers workflow

⚠️ subagent 关键约束

- 不共享 context (避免污染)
- 禁止 it.skip /xit
- 禁 vi.mock 整层 HTTP
- 失败重派 1 次
- 仍失败 → 停·人工介入

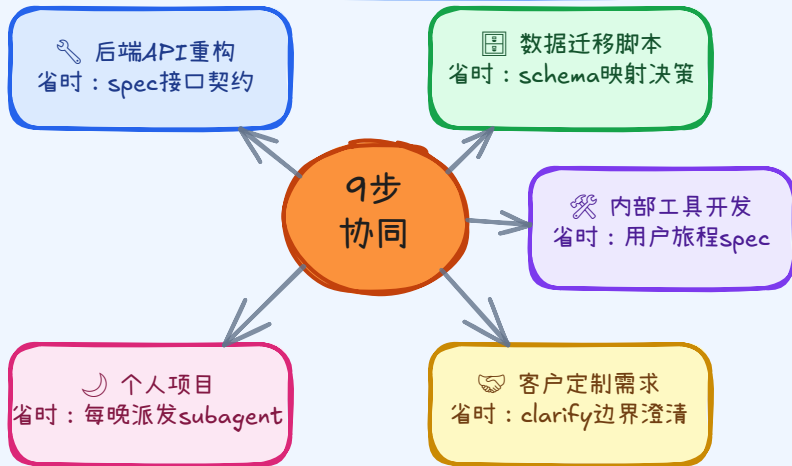
🔄 TDD 红绿循环

每个 subagent 执行:
先写 vitest 测试文件
跑 → 确认 🟡 RED
写最小实现代码
跑 → 确认 🟢 GREEN
重构保持 GREEN

💡 subagent 派发不是「我说12个任务一起干」——spec-kit 已把任务拆到 subagent 可单独执行的粒度·workflow 已定义严格 TDD
前置工作做好了, subagent 只是按部就班。这才是可靠并行, 不是许愿式的「你们都去做」

9步协同不只用于日报Agent · 跨域5场景 · 任何「跨天/跨周」项目都套这套

☆ 同一套9步协同 · 辐射5个跨域场景



⚠ 5场景为本课程延伸应用，非任一工具平台官方提法

📋 现场扫描清单 · 问自己5个问题

我的下一个跨天项目是什么？

它的 spec.md 现在能写出来吗？

能拆成 ≥ 12 个 2-5 min 的子任务？

会先 brainstorming 再 specify 吗？

有没有「裸CC硬干」的跨天项目？

跨天项目用9步 · 跨小时项目用Superpowers · 跨分钟项目用裸CC——这是一把尺子 📏

并发多 Agent 协作开发 · 工程化方法论

用 Superpowers 落地 · 方法论可复用迁移

工程化进阶课 · 不教 NanoClaw · 不教项目细节 · 教可迁移的开发思路

核心价值 · 3条

4个工程化经验 · 拿走就能用
从真实项目萃取，不是纯理论

2个反直觉认知 · 防止你下次走偏
对抗「一切都要并发」等常见陷阱

1张分水岭对照 · 照着自检
判断你的 Agent 系统在哪个层次

案例载体

在 NanoClaw 上做「多平台内容生成」
5 Agent 协作 · 跨仓库 + 后端持久化

本期节奏

定锚 / 经验主线 / 反直觉认知 / 分水岭对照 / 迁移指南

核心价值：不学项目 · 学方法论

🎯 场景定锚 · 我们用「多平台内容生成」做案例载体

在 NanoClaw 平台上 · 1 coordinator + 1 researcher + 3 writer = 5 Agent 协作 · 完成一次内容分发

👤 产品视角 · 4步体验

打开 Dashboard 多平台生成页



输入话题 · 「AI Agent 工程化最新进展」



点触发按钮 · 实时看 5 个 Agent 跑起来



3 平台文章并行生成完成
小红书 / 公众号 / 微博

⚙️ 工程视角 · 5工程维度

📁 跨仓库 · nanoclaw-fork + Dashboard 各自 git 仓库

⚡ 多Agent并发 · Claude SDK + Docker + SQLite 消息总线

📄 真持久化 · content_tasks / content_articles + 长跑 state

🔄 前后端联动 · trigger API + 5s polling + 实时 state 推流

🤝 多Agent协作 · coordinator → 4 sub-agents · handoff 协议

典型「多Agent并发 + 跨仓库 + 前后端联动」三合一场景 · 覆盖你绝大多数真实产品的工程结构

🎯 本期项目实战核心价值

🔑 4个工程化经验·主线

调研先行 → 30 min 换 3 h
决定「要不要开干」

契约先行 → spec.md 明确需求后再写代码
决定「怎么并发不打架」

并行开发 → dispatching-parallel-agents
决定「怎么提效一半」

writing-skills 元方法
→ 跨场景 pattern 才该抽 skill
决定「怎么沉淀经验」

🎯 核心目标

拿到并发多 Agent 需求·你知道用什么思路开局·用什么 skill
卡在哪时该怎么走·而不是瞎猜重来

⚡ 反直觉认知·防走偏

静态 PASS ≠ 动态 PASS

全绿 ≠ 能跑

边修边推进是常态

5 轮收敛 = 真节奏·不是失败

✅ 判断你今天是否带走了

- ☐ 下次开局·先调研还是直接敲？
- ☐ 并发前·写 spec 还是边写边定？
- ☐ 写完代码·真起进程还是 tsc 绿就发？

👤 工程师 vs Vibe Coder · 6维度分水岭

✅ 工程师 ❌ Vibe Coder

拿到需求 先调研 + spec.md 直接敲代码

接口设计 契约先行定义 边写边猜

并发策略 dispatching 派 N 个 串行等结果

验证方式 真起进程测试 tsc 绿 = 通过

踩坑处理 记录分析·逐轮收敛 推倒重来

复盘沉淀 提炼成 skill 留存 下次再说

🚀 不是学一个项目·是建立一套可复用的开发思路·明天就能套到你自己的产品

「不调研直接写代码·一定翻车。调研 30 分钟·省 3 小时推倒重来。」

⚡ 反直觉：你以为调研在浪费时间·真相是省时间的最大杠杆

✗ Vibe Coder 路径·错位心态

拿到需求 → 直接打开 Claude Code 敲

凭直觉假设架构

边写边发现错 → 推倒重来

💬 心态：“写出来再说”

⚠ 反例信号：写一半发现架构错·反复修同处
推倒重写 ≥ 2 次

VS

✓ 工程师路径·正确心态

拿到需求 → 先想清楚要调研的问题

派 sub-agent 扫源码·凭据驱动

写之前已经知道对在哪

💬 心态：“想清楚再动手”

适用：任何“在已有系统加新功能”的需求
拿到需求第一步必须调研·没有例外

📌 不调研的大概率后果

假设 daily-news 是多 Agent → 写 300 行编排代码

发现是单 Agent 7 步 prompt → 推倒 → 损失 3h+

📌 先调研的大概率结果

2 sub-agent 并发扫前后端 → 拿到 brainstorming.md

调研 30 min → 方向已锁定 → 0 推倒重做

🕒 时间杠杆比 1:6 · 30 min 投入 · 撬动 3h+ 浪费

🔍 调研 = 工程师与 Vibe Coder 的分水岭 · 调研不是浪费时间·是工程开发的必要，要做到心中有数

🔍 调研需要想清楚的三个层级（二次开发场景）

复用层

现有架构能复用什么？不能复用什么？

📁 看什么

同类型 feature 实现
现有表结构

案例

daily-news 是单 Agent 7 步 prompt
推翻多 Agent 链路假设

📁 code-survey sub-agent

通信层

前后端通信机制？触发 / 推流路径？

📁 看什么

app/api/ 路由 + 对接
polling 还是 SSE

案例

Dashboard polling hook 5s 拉一次
不需要 WebSocket

📁 dashboard-survey sub-agent

承载层

后端是否支持新场景功能 / 调度需求？

📁 看什么

调度机制 wakeContainer/cron/queue
容器隔离 + 资源限制

案例

NanoClaw wakeContainer 可复用
不用新写调度

📁 主 session 汇总 brainstorming.md

📁 工具：code-survey sub-agent 扫后端 · dashboard-survey sub-agent 扫前端 · 主 session 汇总成 brainstorming.md

📄 产出物：brainstorming.md = 架构决策 + 风险点 + 代码引用 ← 不是脑暴，是结构化结论

✅ 调研充分的 3 个信号

- ❑ 需求疑惑点都有源码引用作答（不是猜）
- ❑ 确定的架构决策都标了「为什么不选其他」
- ❑ 实际开发时存在的风险点都标了「如果发生怎么办」

⚠️ 反例信号（调研不充分）

- 答案是「应该可以复用」（无源码引用）
- 没有备选方案对比 • 没列风险

做到充分调研

超时 = 问题没拆细
→ 再拆一层细粒度后重启

调研后出 brainstorming.md
（含代码引用·非猜测）

⊗ 工程建议：没有 brainstorming.md → 不写第一行代码

Step 1 · 调研问题思路 🔍

🔍 问题1: 现有实现在哪?
「系统X上功能Y: 关键代码
在哪目录? 在哪一行?」

🔗 问题2: 通信机制是什么?
「前后端用什么通信方式?
polling/SSE/WS/queue
能直接复用吗?」

⚡ 问题3: 并发能撑多少?
「现有方案能撑N并发吗?
还是要重新设计扩容?」

📝 你来填写 (针对你的产品):
系统X = 你的后端服务名
功能Y = 你要加的新 feature

→ 根据实际的业务需求扩展!

Step 2 · 并发派 sub-agent ⚡

📦 技能: dispatching-parallel-agents

一条 message 同时派出:
→ subagent A: 扫后端代码
→ subagent B: 扫前端代码

⚠️ 规则:
每个必须写 findings 文件
不能口头汇报!

主 session 等两份都回来
合并分析 → 才做决策

⌚ 效率对比:
串行扫描 → ~ 40 min
并发扫描 → ~ 20 min

提效整整一半 🏹
下次照单全收!

Step 3 · 汇总 brainstorming.md 📄

📄 brainstorming.md 包含:

多个技术决策
→ 每条附「为什么不选X」
→ 每条带源码引用

多个风险点
→ 每条附「如果发生怎么办」
→ 每条带源码引用

格式: src/xxx.py 第 N 行
(精确到行号!)

✅ 完成判断 (3 个 □):
□ 每个 决策都有源码引用
□ 每个 风险都有应对方案
□ 团队 review 后没人问
「你怎么知道?」
三勾全上 → 开干! ☺

🕒 调研报告不是文档 · 是研发开工许可证

S01 · 并发原理 + 上下文经济学 — dispatching-parallel-agents 是什么 · 为什么

⚠ 最大误解 · Skill 不实现任何并发机制——Claude Code Harness 原生的 Agent(...) 工具才是真正的引擎

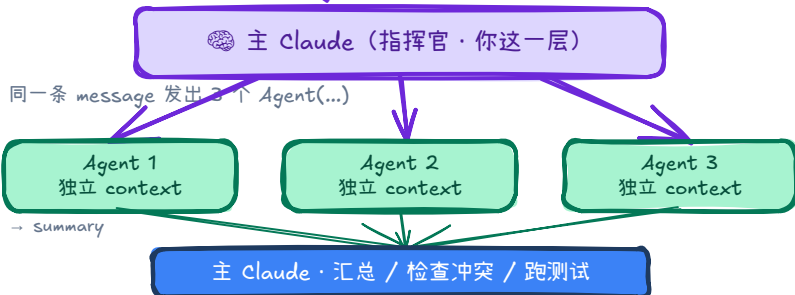
✗ Skill ≠ 运行时

✗ Skill ≠ 并发框架

✓ Skill = 方法论 / 决策图

📄 需求文档 = Skill | 🌸 Jira = Harness

并发架构 Fan-Out + Convergence



核心设计动机 · SKILL.md 原话

"They should never inherit your session's context or history — you construct exactly what they need. This also preserves your own context for coordination."

翻译：子 agent 永远不应继承你 session 的 context——你为它精确构造所需信息。这也保留了你自己的 context 用于协调工作。

上下文窗口是稀缺资源 · 串行 vs 并行 3 维对比

维度	✗ 串行做 3 任务	✓ 并行 dispatch 3 agents
上下文累积	主 context 累积全部探索过程	主 context 只保留 3 个 summary
会话状态	容易爆炸 · 需要 /compact	主 Claude 保持指挥官清爽视角
耗时	慢——串行时间累加	快——按最长那个算

🕒 省时间 · 并行 vs 串行

💰 省 token · 独立 ctx 不污染

🧠 省心智带宽 · 主 Claude 不被噪音淹没 · 保持指挥官清爽

类比：📄 需求文档 = Skill | 🌸 Jira = Harness (各司其职)

vs 手动开 3 个 tmux 并行的本质区别

✗ 手动 copy-paste 各窗口结果

✗ 注意力在 3 个 context 间反复切换

✓ Skill 模式：主 Claude 拿到 3 份 summary 自动汇总判断

dispatch 触发判定 · 何时派发并行 Agent

dispatch 触发条件 =

失败/任务数 ≥ 2

AND 彼此独立·互不影响

AND 不需要共享状态

AND 修一个不会顺带修好另一个

⚠ 任一条不满足都不应 dispatch · 会出乱子

✅ 典型触发场景 · 5 例

多文件测试失败·根因不同

3 测试挂：超时/事件 bug/异步等待 · 彼此正交

多个独立子系统坏

前端A/后端B/WorkerC 各自有独立 bug

多个独立调研

同时调研 5 个竞品/库/仓库 · 互不影响

跨文件批量改造

10 个 .test.ts 从 mock → real db · 每文件独立

多语言/多平台并行打包

iOS/Android/Web 三套独立构建

✅ dispatch 前自检 3 问

🔍 根因是否独立？

🔗 修复路径是否互不影响？

🔗 是否需要共享中间状态？

❌ 不该用场景 · 4 例

关联失败·修一处全好

核心 util 改坏 → 10 测试挂 · 修 util 一次性全好

不要派 10 个 agent

开发过程中探索性 debug, 注意：不是调研阶段

不知道哪里坏 · 需先看整体再说 · 并行只会浪费

共享文件/同一资源

3 agent 都要改 package.json → 冲突, 无法并行

强顺序依赖

A 的输出是 B 的输入 · 必须串行

⚡ 一句话总结

dispatch 不是「并行更快」

而是「独立任务才该并行」

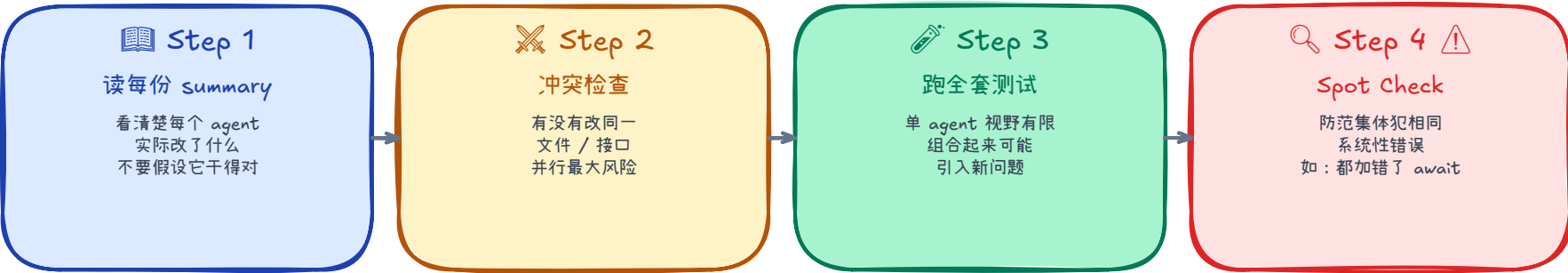
错误并行 = 比串行更慢 · 还引入冲突

dispatch = 独立性判断, 不是速度优化

FCSO 4 要素 — 每个 sub-agent prompt 必须含这四点，缺一不可

要素	含义	反例 (❌) → 正例 (✅)	⊘ 反模式 · 绝对禁止
F Focused	单一问题域	❌ 修所有测试 → ✅ 只修 agent-tool-abort.test.ts	⊘ 仅放大 timeout · 不找真正根因
C Context	自包含上下文	❌ 修这个 race condition → ✅ 贴具体错误信息 + 测试名	⊘ 随手 refactor · 超出 scope
S Scope	边界约束	❌ 随便改 → ✅ 不改生产代码·只调测试·不要无脑加 timeout·找真正根因	⊘ 返回 done · 无契约信息
O Output	输出契约	❌ 修好 → ✅ 返回根因 + 改了什么 + 文件列表	✅ 4 要素都到位 = 高质量 prompt

Verification 4 步 — dispatch 后必须做，跳过这步 = 把并行用成了「naive 并行」



FCSO 决定输入质量 · Verification 决定输出质量 — 两端把关，并行才真正比串行强

dispatch 不是终点 · Verification 才是质量分水岭

「明确契约规则，解决并发冲突· spec.md 是 N 个 sub-agent 看的同一份合同。」

⚡ 反直觉：不是「先写代码再补文档」·是「先把契约钉死·才有资格并发」

📦 spec-kit · 4 件套

spec.md → 1 功能需求 + 验收标准

clarify.md → 拍板决策

plan.md → 模块架构图 + 数据流

tasks.md → N tasks + 文件域 + 依赖

4 件 = 并发的全部契约

📄 N 个 sub-agent · 共看一份合同



🔒 谁也不能私改·改了通知所有人

🎯 适用·任何并发开发场景

多 Agent 并发开发同一 feature

多人协作·分模块开发

多模块/微服务接口对接

任何「需要对齐才能合作」的场景

有并发 = 先有契约

specify → clarify → plan → tasks → worktree → 派 subagent → review → finish

⚠️ 没有契约的代价情况实例：

eg: platform 字段类型不一致 → 5 个文件要改 → 耗时 $\times 3$

极端：4 sub-agent 各自定义 type → 全盘重做

review 阶段才发现 → 返工成本 $\times 3$

✅ 有契约·并发才有意义

spec.md 钉死接口 → 每个 sub-agent 读同一份

并发完成 → review 走过场

接口对齐成本：0

❌ 边写边定 → review 才发现 → 修复成本指数级 ✅ 先钉死 → 并发开干 → review 走过场

🔑 契约 = N 个 sub-agent 共同的 source of truth · 谁也不能私改·改了通知所有人

📄 没有契约的并发·就是混乱 · spec.md 是合同·不是文档

1 条 message · N 个 subagent 同时跑 · 契约钉死是前提 · 并发才能不打架

🔄 1 message → N subagent · 并发派发机制



独立 context · 独立 sandbox · 互不阻塞 · 主 session 统一汇总

⚡ 反直觉 + Skill 定位

大多数人只在「开发」阶段并发 · 忘了「调研」也能并发 → 提效一半

Claude Code 主 session 在 1 条 message 内并发派发多个 Task subagent
互不阻塞 · 独立 context · 独立 sandbox

主 session 在 1 条 message 内连续调用 N 次 Task tool

Claude Code 异步
同时启动 N 个 subagent

每个 subagent
独立 context
工具 · sandbox

主 session 等所有 subagent 回来
统一汇总

🔍 3 大适用场景

调研类 | 多源头独立扫描
code+dashboard
开发类 | 多域文件独立修改
4 sub 各写各目录
评估类 | 多角度独立判断
性能·安全·兼容各自打分

⚠️ 前提条件 (缺一不可)

文件域不重叠
不同 sub 写不同目录
共享只读契约
spec.md 不可写
主 session 统一 commit
sub 内部不 commit

⚡ 并发不是技巧 · 是契约钉死之后的自然结果 · 不会并发 = 工程师效率折半

⚡ 并发实战：3 条真实技巧 vs 3 个必须躲的反模式

✓ 3 条真实使用技巧（附为什么）

派之前一定写完 prompt 文件

为什么：subagent 无全局视野，只能靠 prompt + spec 推断

明确写出每个 subagent 的文件域

为什么：并发写同一文件 → git 冲突 / 后写覆盖先写

每个 subagent 必须写 findings 文件

为什么：防偷工 · 主 session 有据可查 · 凭据可追溯

⊗ 3 个反模式（附正确做法）

任务 B 依赖任务 A 的运行时输出

正确：串行（A 完成 → B 启动）

多个 subagent 写同一 mutable 文件

正确：串行 · 拆分文件域 · 或一写一审

一个 sub-agent 失败 · 其他产出全部无效

正确：先串行完成基础，再并发扩展

🎯 技巧记不住没关系 · 3 反模式必须躲！

🔗 怎么把「并发开发」正确应用到场景中 · 可行性 3 问 + 落地模板

🧐 判断你的场景能不能并发 · 3 问框架

问1·依赖问题：任务间有运行时依赖吗？

有依赖

→ 串行

↓ 无依赖

问2·文件域问题：文件能拆开吗？

写同一文件

→ 串行 / 重拆

↓ 各写各自的目录

问3·失败回收问题：失败策略想清楚了吗？

失败不可隔离

→ 先串行基础

↓ 失败可隔离

✅ 3问全YES

→ dispatching-parallel-agents

❌ 任一NO

→ 串行 / 重拆

🔗 用法对照

调研阶段 · 派 2 个 → 后端 src/ + 前端 app/ (各自独立·互不干扰)

开发阶段 · 派 4 个 → DB / .env / API / 前端组件 (4 个文件域不重叠)

每次派发：一条 message · N 次 Task tool 并行触发 (不要分多条 message)

📄 落地模板 · 一条 message 的结构

我要派 N 个并发 subagent · 各自任务如下：

Sub-A: specs/sub-a.md → src/moduleA/

Sub-B: specs/sub-b.md → src/moduleB/

每个 sub 必须写：findings/sub-X.md

全部回来后 → verify + commit

→ 一条 msg · N 次 Task tool 并行触发

派发前自检 · 3 个 □

□ 我有 spec.md 当共享契约吗？

□ N 个 subagent 文件域真不重叠吗？

□ 失败回收策略想清楚了吗？

全 □ 打勾 → 放心并发 ✅

有 □ 未勾 → 重新拆分任务 🔄

不会并发不可怕 · 乱并发才致命

写 Skill = TDD 应用到流程文档上

这不是比喻·是字面意义——整个 skill 把 RED-GREEN-REFACTOR 循环原封不动套到「写文档」上

TDD 概念	Skill 创作
测试用例	Pressure scenario (压力场景)
生产代码	SKILL.md 文档
RED · 测试失败	没 skill · agent 违规基线行为
GREEN · 测试通过	加上 skill 后 · agent 合规
REFACTOR · 重构	堵漏 plug loopholes · 保持合规

NO SKILL WITHOUT A FAILING TEST FIRST

没看到 agent 失败前·你不知道 skill 该教什么

3 大核心问题·skill 为「未来 AI」设计而不是给人看

🔍 未来 agent 找得到你的 skill 吗? → 需要 CSO (Claude Search Optimization)

🕒 找到后会真按 skill 做吗? → 需要 RED-GREEN 验证

🛡️ 遇到压力会找借口绕过吗? → 需要 REFACTOR 堵漏

Description 反直觉规则·最被忽视的一条·必须单独强调

✗ BAD 描述了 workflow → Claude 走捷径跳过 SKILL.md 正文

description: Use when executing plans -
dispatches subagent per task
with code review between tasks

↑ 包含了 workflow 细节

→ Claude 直接按这段做·跳过正文!

→ "两阶段 review" 变 "一次 review"

✓ GOOD 只描述触发条件·workflow 留在正文

description: Use when executing
implementation plans with
independent tasks in the
current session

↑ 只说触发时机·不提 workflow

→ Claude 读完 description 进正文

→ 完整执行两阶段 review

LLM 本质: description 字段优先注入系统提示·权重覆盖正文 —— 这是整个 skill 揭示的最深刻洞察

写 Skill 不是「写文档」·是「给未来 AI 写 TDD 测试用例」

—— Prompt Engineering 从经验主义 → 可工程化的关键跃迁

5 阶段 RED-GREEN-REFACTOR + 4 种 Skill 类型测试策略



⚠ 禁止批量造 skill · 单 skill 走完 5 阶段才能写下一个

4 种 Skill 类型 · 决定你在 RED 阶段如何设计 pressure scenario (选错类型 = 测试方向错)

类型	例子	测试方法	成功判据
Discipline 纪律性	TDD verification- before-completion	学术题 + 压力场景 (pressure scenario) time / sunk cost / authority / exhaustion 四压力组合	极限压力下 agent 仍合规
Technique 技法	condition-based- waiting	应用场景 + 边缘场景 + 缺信息场景	agent 能套用到新场景
Pattern 模式	flatten-with-flags	识别场景 + 反例场景	agent 能判断「何时不该用」
Reference 参考	API 文档	检索场景 + 应用场景 + 覆盖率测试	agent 能找到并正确使用

💡 这个分类比「Skill 是什么」更重要
——它决定了 RED 阶段怎么设计 pressure scenario
选错类型 = 测试方向错

5 阶段是工艺 · 4 种 Skill 是分类
两者匹配才能造出真正可用的 skill

Flowchart 使用纪律：决策非显然才配画图

✓ 只在以下情况用 graphviz 流程图

- 非显然的决策点
- 容易过早停止的循环
- "A vs B" 选择

✗ 绝不在以下情况用

- 参考资料 → 用表格
- 代码示例 → 用 markdown 代码块
- 线性指令 → 编号列表 · 无语义标签 (step1)

💡 迁移价值：很多人滥用流程图凑工程感 → 这个 skill 直接说「决策非显然才配画图」

4 条容易忽略的硬规矩

🔪 @filename = 核武器
立即压入全文·烧 200k+
token·禁止跨 skill 引用
→ 用纯字符串引用

⏏ STOP 规则
写完一个 skill
必须跑完测试再写下一个
不准批量造 skill

🎯 不要多语言示例
1 个高质量 > 5 个平庸
agent 会自己翻译
别浪费 token

📖 不写叙事
"2025-10-03 发现..."
是反模式
skill = 参考手册·非事故复盘

三层抽象 · (自下而上·逐层递进)

对抗设计

🛡️ 第三层·工程方法 Rationalization Table + Red Flags + Loophole 堵漏 → Prompt Engineering 从经验主义 → 可工程化的关键跃迁

机制

⚙️ 第二层·机制揭示 description 是注入系统提示的高权重文本 → 用这一观察把「LLM 提示词权重 / 上下文经济学」讲深一层

🔄 第一层·认知颠覆 「写 skill 不是写文档·是给未来 AI 写测试驱动开发」 → 从「写得清楚」升级到「可验证·可对抗·可演化」

「真踩·真抽·真验证·不是预判。没有 failing test 凭据的 skill 是空中楼阁。」

反直觉

95% 的 skill 预判都猜错
只有真踩过 ≥ 2 次同类坑
才知道哪个是跨场景 pattern

本质·1 句

writing-skills 是元方法
是「何时该写 skill」
的判断框架

Skill 定位

发现跨场景重复 LLM 行为坑
→ RED-GREEN-REFACTOR 闭环
抽成可复用 skill
→ 下次不重新踩坑

RED → GREEN → REFACTOR 闭环

RED · 写 failing test

记录「坑长什么样」
留作回归测试凭据
必须基于真凭据·不能编

REFACTOR · 真验证

重跑 failing test·对比数据
没有数据对比 = 没修

GREEN · 写 skill

让 failing test 通过
三戒律自检·真改 LLM 行为
AI 禁代写 SKILL.md 内容

✓ 该抽 skill 的 3 个信号

跨场景重复 (≥ 2 个不同 agent)
文档写明 LLM 还是会犯
有真凭据数据支撑 (log/DB)
(三者都满足才抽)

✗ 反例·不该抽

单场景一次性 bug → 写进 CLAUDE.md
配置文件错 → 不是 skill 该解决
凭直觉预判 → 95% 错

发现跨场景重复的 LLM 行为坑
→ RGR 闭环 → 可复用 skill
→ 让下次不重新踩坑

✎ 真实触发例 · 3 writer 对照实验

✓ weibo writer → 真调 Bash → 真 INSERT → DB 1 行

✗ xhs writer → 输出「已完成 ✓」→ 跳过 Bash → DB 0 行

✗ gzh writer → 输出「文章已交付」→ 跳过 Bash → DB 0 行

→ 跨 2 个 writer 重复同一 pattern → 抽 skill 候选: action-vs-narrative-separation

🕵️ 真踩才知道哪个值得抽·没踩过的坑·不该抽成 skill

⚖️ 该抽 vs 不该抽 · writing-skills 三戒律

✗ 不该抽 skill (凭据不足)

- 单场景小问题 → CLAUDE.md 一句话搞定
- 配置文件写错 → 是 config 问题, 不是行为问题
- 一次性 bug, 修了不重现 → 无回归价值
- CLAUDE.md 一句话能解决 → 不需要 skill
- 无 failing test 凭据 → 凭直觉预判 95% 错

✓ 该抽 skill (凭据充分)

- 跨场景重复 pattern → 多场景验证过
- LLM 行为习惯 (非配置问题) → 文档管不住
- 多次开发都会遇到 → 有复利价值
- 即使文档清楚也会犯 → 需要 CSO 强触发
- 有真凭据数据支撑 → RED test 跑得起来

vs

写下来 1 句能解决 → CLAUDE.md | 写下来 1 页都解决不了的行为模式 → skill

戒律 1 · description 句式

- 🔑 Use when... 开头 · 第三人称 · ≤500 chars
- 🔍 wc -c description.txt
- ✗ 第一人称 I notice... → CSO 匹配混乱

戒律 2 · prose 字数

- 🔑 整体 prose (去 frontmatter) ≤ 500 词
- 🔍 wc -w skill-body.md
- ✗ 长篇 workflow 总结 → Claude 读不完

戒律 3 · 内容结构

- 🔑 只写「识别 → 决策规则」
- 🔍 找「步骤 N」段 → 应该 0 个
- ✗ 「5 步流程」→ 是手册不是 skill

⚠️ 不过三戒律的 skill = 污染 = 反作用

🔍 把「writing-skills 元方法」用到日常开发的规范

📖 识别「跨场景重复 Pattern」的 3 个信号 · 全亮才抽

🕒 信号 1 · 跨场景重复

? 这个坑在 ≥ 2 个不同 agent / 不同场景出现了吗?

- ✓ 正例: 长文 writer 跨 xhstgz
- 2 个场景都跳过 Bash
- ✗ 反例: 只在 1 个 agent 出现

🕒 信号 2 · 文档无效

? CLAUDE.md 已写明 · LLM 还是会犯吗?

- ✓ 正例: 写了 MANDATORY ORDER · weibo 听 / xhstgz 不听
- ✗ 反例: 还没写文档就抽 → 跳步骤

🕒 信号 3 · 真凭据充分

? 有 log / DB / 输出对比的硬数据吗?

- ✓ 正例: DB 0 行 vs 1 行对比表
- ✗ 反例: 「好像 Claude 经常这样」
印象 ≠ 凭据

✓ 3 信号全亮 → 走 RED-GREEN-REFACTOR 闭环抽 Skill ⚠️ 任一不满足 → 一句话写进 CLAUDE.md · 继续观察凑齐凭据

🐱 起名规范 · 名字 = 跨场景 Pattern 的本质

✗ 描述项目 / 角色 / 实现

my-writer-skill

nanoclaw-bash-fix

xhs-fix

✓ 描述 Pattern 的本质

action-vs-narrative-sep

tool-call-after-longform

silent-skip-on-completion

🔄 RED-GREEN-REFACTOR 闭环模板 · 照着填

🕒 RED

failing-test.md

+ 真凭据数据

🕒 GREEN

SKILL.md

三戒律自检通过

🕒 REFACTOR

重跑 test
数据对比
验证修复

🧠 经验沉淀的纪律 = 工程师的护城河 · 不预判 · 不替 Claude 决定 · 只在真凭据驱动下抽 Skill

「真实开发不是一次完美·是边修边推进。每轮发现一个新坑·就是真进展。」

反直觉：一次跑通是 Vibe Coder 的幻觉·真实工程师从来都是 N 轮收敛

5 轮联调数据·每轮修 1 个根因·节奏在收敛·总耗时 ~2h

坑1

轮 1 · ~30 min

容器路径不存在
→ 加 additionalMounts

坑2

轮 2 · ~25 min

挂载前缀错
→ 改 5 个 CLAUDE.local 路径

坑3

轮 3 · ~20 min

消息没路由
→ 加 XML 强制要求

坑4

轮 4 · ~15 min

session 残留
→ 删 sessions + outbound.db

坑5

轮 5 · ~30 min

长文跳工具
→ MANDATORY ORDER

心态对照·同一件事，两种反应

❌ 反例·心态错位

- 跑一次没过 → 怀疑架构 → 推倒重写
- 一遇坑 → 「是不是我整体方案错了」
- 修了 2 轮还没好 → 不看 log 看情绪

同一个坑修 3 次还出现 → root assumption 没找到
每轮耗时变长 → 调试方法错，不是问题难
越修越多坑 → 改错了又改回，缺凭据回归

vs

✅ 正例·心态对位

- 发现新坑 → 标记笔记 → 下次能给答案
- 不怀疑大方向 → 只问这一轮根因是什么
- 修了 5 轮还没好 → 查 root assumption

每轮的坑都不同 → 没在原地打转
每轮耗时变短 → 学到了不重犯
每轮根因更深 → 越来越近 root cause

◎ N 轮收敛 = 工程化的真节奏·每个新坑都是真进展·不是失败

⚖️ 工程师 vs Vibe Coder · 6 维度分水岭对照

维度	⚡ Vibe Coder 做法	🔗 工程师做法	🏠 后果差
🎯 拿到需求	直接打开 Claude Code 敲	先调研 3 问 派 sub-agent 扫源码	V/C: 推倒重来 3h vs 工程师: 30min 对线
📄 接口设计	边写边定 边发现边补	spec.md 钉死 5 项 再动手写	V/C: review 修复 ×3 vs 工程师: review 走过场
⚡ 并发开发	不并发 / 乱并发 同文件冲突	契约+文件域 一条 msg 派 N agent	V/C: 0 提效或负提效 vs 工程师: 省 50-75%
🔍 验证	tsc 绿就发布	起进程·点按钮 查 state·重启验证	V/C: 线上才发现 vs 工程师: 上线前发现
🌀 踩坑心态	是失败·是耻辱 → 怀疑自己	进展+1 经验 → 是下次答案	V/C: 心态崩→放弃 vs 工程师: 收敛→闭环
📚 复盘沉淀	修完就忘	抽 pattern → skill 沉淀到团队	V/C: 同坑踩 N 次 vs 工程师: 踩 1 次终身免疫

☆ 工程师和 Vibe Coder 的差别·不是技术·是方法论

可复用的工程方法论

多 Agent 协作需求

第一动作

调研 3 问 · 2 sub-agent 并发扫描源码
(前端 + 后端)

🔗 dispatching-parallel-agents

□ brainstorming.md 4 决策 + 5 风险
引用来源都齐了吗？

⚠️ 跳过 → 推倒重来 ≥ 3 h

任何并发场景

第一动作

5 项契约清单先钉死
再动一行代码

🔗 spec-kit 9 步流程

□ spec.md 5 项全钉了吗？
type+API+CLI+文件域+状态枚举

⚠️ 跳过 → 修复成本 ×3

反复犯的 LLM 坑

第一动作

3 信号判断 · 满足条件就抽 skill
(≥2场景+文档管不住+有凭据)

🔗 writing-skills 元方法

□ ≥2 场景重复 + 文档管不住
+ 有真凭据？

⚠️ 跳过 → 同一坑下次还踩

📦 默认装备 → spec-kit · dispatching-parallel-agents · writing-skills → 不装这 3 个，你还在用 Vibe Coder 方法论

本质迁移 🔄

案例场景只是演示载体 · 真正可复用的完整开发思路

💠 微服务 → spec-kit + 并发

💠 AI Agent → 调研先行 + 沉淀

🌐 SaaS → 整套都用 (最佳收益)